

This is CS50

CS50's Introduction to Computer Science

OpenCourseWare

Donate  (<https://cs50.harvard.edu/donate>)

David J. Malan (<https://cs.harvard.edu/malan/>)
malan@harvard.edu

 (<https://www.facebook.com/dmalan>)  (<https://github.com/dmalan>) 
(<https://www.instagram.com/davidjmalan/>)  (<https://www.linkedin.com/in/malan/>) 
(<https://www.reddit.com/user/davidjmalan>)  (<https://www.threads.net/@davidjmalan>)
 (<https://twitter.com/davidjmalan>)

Speller

Problem to Solve

For this problem, you'll implement a program that spell-checks a file, a la the below, using a hash table.

Demo

```
tempore

WORDS MISPELLED:      30
WORDS IN DICTIONARY:  143091
WORDS IN TEXT:        7573
TIME IN load:         0.05
TIME IN check:        0.01
TIME IN size:         0.00
TIME IN unload:       0.00
TIME IN TOTAL:        0.06

$
```

Recorded with **asciinema**

Distribution Code

For this problem, you'll extend the functionality of code provided to you by CS50's staff.

▼ Download the distribution code

Log into [cs50.dev \(https://cs50.dev/\)](https://cs50.dev/), click on your terminal window, and execute `cd` by itself. You should find that your terminal window's prompt resembles the below:

```
$
```

Next execute

```
wget https://cdn.cs50.net/2024/fall/psets/5/speller.zip
```

in order to download a ZIP called `speller.zip` into your codespace.

Then execute

```
unzip speller.zip
```

to create a folder called `speller`. You no longer need the ZIP file, so you can execute

```
rm speller.zip
```

and respond with “y” followed by Enter at the prompt to remove the ZIP file you downloaded.

Now type

```
cd speller
```

followed by Enter to move yourself into (i.e., open) that directory. Your prompt should now resemble the below.

```
speller/ $
```

Execute `ls` by itself, and you should see a few files and folders:

```
ictionaries/  dictionary.c  dictionary.h  keys/  Makefile  speller.c  speller5
```

If you run into any trouble, follow these same steps again and see if you can determine where you went wrong!

Background

Given the many files in this program, it's important to read this section in its entirety before starting. You'll then know what to do and how to do it!

Theoretically, on input of size n , an algorithm with a running time of n is “asymptotically equivalent,” in terms of O , to an algorithm with a running time of $2n$. Indeed, when describing the running time of an algorithm, we typically focus on the dominant (i.e., most impactful) term (i.e., n in this case, since n could be much larger than 2). In the real world, though, the fact of the matter is that $2n$ feels twice as slow as n .

The challenge ahead of you is to implement the fastest spell checker you can! By “fastest,” though, we’re talking actual “wall-clock,” not asymptotic, time.

In `speller.c`, we’ve put together a program that’s designed to spell-check a file after loading a dictionary of words from disk into memory. That dictionary, meanwhile, is implemented in a file called `dictionary.c`. (It could just be implemented in `speller.c`, but as programs get more complex, it’s often convenient to break them into multiple files.) The prototypes for the functions therein, meanwhile, are defined not in `dictionary.c` itself but in `dictionary.h` instead. That way, both `speller.c` and `dictionary.c` can `#include` the file. Unfortunately, we didn’t quite get around to implementing the loading part. Or the checking part. Both (and a bit more) we leave to you! But first, a tour.

Understanding

dictionary.h

Open up `dictionary.h`, and you'll see some new syntax, including a few lines that mention `DICTIONARY_H`. No need to worry about those, but, if curious, those lines just ensure that, even though `dictionary.c` and `speller.c` (which you'll see in a moment) `#include` this file, `clang` will only compile it once.

Next notice how we `#include` a file called `stdbool.h`. That's the file in which `bool` itself is defined. You've not needed it before, since the CS50 Library used to `#include` that for you.

Also notice our use of `#define`, a "preprocessor directive" that defines a "constant" called `LENGTH` that has a value of `45`. It's a constant in the sense that you can't (accidentally) change it in your own code. In fact, `clang` will replace any mentions of `LENGTH` in your own code with, literally, `45`. In other words, it's not a variable, just a find-and-replace trick.

Finally, notice the prototypes for five functions: `check`, `hash`, `load`, `size`, and `unload`. Notice how three of those take a pointer as an argument, per the `*`:

```
bool check(const char *word);
unsigned int hash(const char *word);
bool load(const char *dictionary);
```

Recall that `char *` is what we used to call `string`. So those three prototypes are essentially just:

```
bool check(const string word);
unsigned int hash(const string word);
bool load(const string dictionary);
```

And `const`, meanwhile, just says that those strings, when passed in as arguments, must remain constant; you won't be able to change them, accidentally or otherwise!

dictionary.c

Now open up `dictionary.c`. Notice how, atop the file, we've defined a `struct` called `node` that represents a node in a hash table. And we've declared a global pointer array, `table`, which will (soon) represent the hash table you will use to keep track of words in the dictionary. The array contains `N` node pointers, and we've set `N` equal to `26` for now, to match with the default `hash` function as described below. You will likely want to increase this depending on your own implementation of `hash`.

Next, notice that we've implemented `load`, `check`, `size`, and `unload`, but only barely, just enough for the code to compile. Notice too that we've implemented `hash` with a sample algorithm based on the first letter of the word. Your job, ultimately, is to re-implement those functions as cleverly as possible so that this spell checker works as advertised. And fast!

`speller.c`

Okay, next open up `speller.c` and spend some time looking over the code and comments therein. You won't need to change anything in this file, and you don't need to understand its entirety, but do try to get a sense of its functionality nonetheless. Notice how, by way of a function called `getusage`, we'll be "benchmarking" (i.e., timing the execution of) your implementations of `check`, `load`, `size`, and `unload`. Also notice how we go about passing `check`, word by word, the contents of some file to be spell-checked. Ultimately, we report each misspelling in that file along with a bunch of statistics.

Notice, incidentally, that we have defined the usage of `speller` to be

```
Usage: ./speller [dictionary] text
```

where `dictionary` is assumed to be a file containing a list of lowercase words, one per line, and `text` is a file to be spell-checked. As the brackets suggest, provision of `dictionary` is optional; if this argument is omitted, `speller` will use `dictionaries/large` by default. In other words, running

```
./speller text
```

will be equivalent to running

```
./speller dictionaries/large text
```

where `text` is the file you wish to spell-check. Suffice it to say, the former is easier to type! (Of course, `speller` will not be able to load any dictionaries until you implement `load` in `dictionary.c`! Until then, you'll see `Could not load.`)

Within the default dictionary, mind you, are 143,091 words, all of which must be loaded into memory! In fact, take a peek at that file to get a sense of its structure and size. Notice that every word in that file appears in lowercase (even, for simplicity, proper nouns and acronyms). From top to bottom, the file is sorted lexicographically, with only one word per line (each of which ends with `\n`). No word is longer than 45 characters, and no word appears more than once. During development, you may find it helpful to provide `speller` with a `dictionary` of your own that

contains far fewer words, lest you struggle to debug an otherwise enormous structure in memory. In `dictionaries/small` is one such dictionary. To use it, execute

```
./speller dictionaries/small text
```

where `text` is the file you wish to spell-check. Don't move on until you're sure you understand how `speller` itself works!

Odds are, you didn't spend enough time looking over `speller.c`. Go back one square and walk yourself through it again!

`texts/`

So that you can test your implementation of `speller`, we've also provided you with a whole bunch of texts, among them the script from *La La Land*, the text of the Affordable Care Act, three million bytes from Tolstoy, some excerpts from *The Federalist Papers* and Shakespeare, and more. So that you know what to expect, open and skim each of those files, all of which are in a directory called `texts` within your `pset5` directory.

Now, as you should know from having read over `speller.c` carefully, the output of `speller`, if executed with, say,

```
./speller texts/lalaland.txt
```

will eventually resemble the below.

Below's some of the output you'll see. For information's sake, we've excerpted some examples of "misspellings." And lest we spoil the fun, we've omitted our own statistics for now.

MISSPELLED WORDS

```
[...]  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAHT  
[...]  
Shangri  
[...]  
fianc  
[...]  
Sebastian's  
[...]
```

```
WORDS MISSPELLED:  
WORDS IN DICTIONARY:  
WORDS IN TEXT:  
TIME IN load:  
TIME IN check:
```

```
TIME IN size:
TIME IN unload:
TIME IN TOTAL:
```

`TIME IN load` represents the number of seconds that `speller` spends executing your implementation of `load`. `TIME IN check` represents the number of seconds that `speller` spends, in total, executing your implementation of `check`. `TIME IN size` represents the number of seconds that `speller` spends executing your implementation of `size`. `TIME IN unload` represents the number of seconds that `speller` spends executing your implementation of `unload`. `TIME IN TOTAL` is the sum of those four measurements.

Note that these times may vary somewhat across executions of `speller`, depending on what else your codespace is doing, even if you don't change your code.

Incidentally, to be clear, by “misspelled” we simply mean that some word is not in the `dictionary` provided.

Makefile

And, lastly, recall that `make` automates compilation of your code so that you don't have to execute `clang` manually along with a whole bunch of switches. However, as your programs grow in size, `make` won't be able to infer from context anymore how to compile your code; you'll need to start telling `make` how to compile your program, particularly when they involve multiple source (i.e., `.c`) files, as in the case of this problem. And so we'll utilize a `Makefile`, a configuration file that tells `make` exactly what to do. Open up `Makefile`, and you should see four lines:

1. The first line tells `make` to execute the subsequent lines whenever you yourself execute `make speller` (or just `make`).
2. The second line tells `make` how to compile `speller.c` into machine code (i.e., `speller.o`).
3. The third line tells `make` how to compile `dictionary.c` into machine code (i.e., `dictionary.o`).
4. The fourth line tells `make` to link `speller.o` and `dictionary.o` in a file called `speller`.

Be sure to compile `speller` by executing `make speller` (or just `make`). Executing `make dictionary` won't work!

Specification

Alright, the challenge now before you is to implement, in order, `load`, `hash`, `size`, `check`, and `unload` as efficiently as possible using a hash table in such a way that `TIME IN load`, `TIME IN check`, `TIME IN size`, and `TIME IN unload` are all minimized. To be sure, it's not obvious what it even means to be minimized, inasmuch as these benchmarks will certainly vary as you feed `speller` different values for `dictionary` and for `text`. But therein lies the challenge, if not the fun, of this problem. This problem is your chance to design. Although we invite you to minimize space, your ultimate enemy is time. But before you dive in, some specifications from us.

- You may not alter `speller.c` or `Makefile`.
- You may alter `dictionary.c` (and, in fact, must in order to complete the implementations of `load`, `hash`, `size`, `check`, and `unload`), but you may not alter the declarations (i.e., prototypes) of `load`, `hash`, `size`, `check`, or `unload`. You may, though, add new functions and (local or global) variables to `dictionary.c`.
- You may change the value of `N` in `dictionary.c`, so that your hash table can have more buckets.
- You may alter `dictionary.h`, but you may not alter the declarations of `load`, `hash`, `size`, `check`, or `unload`.
- Your implementation of `check` must be case-insensitive. In other words, if `foo` is in dictionary, then `check` should return true given any capitalization thereof; none of `foo`, `fo0`, `f0o`, `f00`, `Foo`, `Fo0`, `F0o`, and `F00` should be considered misspelled.
- Capitalization aside, your implementation of `check` should only return `true` for words actually in `dictionary`. Beware hard-coding common words (e.g., `the`), lest we pass your implementation a `dictionary` without those same words. Moreover, the only possessives allowed are those actually in `dictionary`. In other words, even if `foo` is in `dictionary`, `check` should return `false` given `foo's` if `foo's` is not also in `dictionary`.
- You may assume that any `dictionary` passed to your program will be structured exactly like ours, alphabetically sorted from top to bottom with one word per line, each of which ends with `\n`. You may also assume that `dictionary` will contain at least one word, that no word will be longer than `LENGTH` (a constant defined in `dictionary.h`) characters, that no word will appear more than once, that each word will contain only lowercase alphabetical characters and possibly apostrophes, and that no word will start with an apostrophe.
- You may assume that `check` will only be passed words that contain (uppercase or lowercase) alphabetical characters and possibly apostrophes.
- Your spell checker may only take `text` and, optionally, `dictionary` as input. Although you might be inclined (particularly if among those more comfortable) to “pre-process” our default dictionary in order to derive an “ideal hash function” for it, you may not save the output of any such pre-processing to disk in order to load it back into memory on subsequent runs of your spell checker in order to gain an advantage.

- Your spell checker must not leak any memory. Be sure to check for leaks with `valgrind`.
- **The hash function you write should ultimately be your own, not one you search for online.**

Alright, ready to go?

- Implement `load`.
- Implement `hash`.
- Implement `size`.
- Implement `check`.
- Implement `unload`.

Hints

Click the below toggles to read some advice!

▼ Implement `load`

Complete the `load` function. `load` should load the dictionary into memory (in particular, into a hash table!). `load` should return `true` if successful and `false` otherwise.

Consider that this problem is just composed of smaller problems:

1. Open the dictionary file
2. Read each word in the file
 1. Add each word to the hash table
3. Close the dictionary file

Write some pseudocode to remind yourself to do just that:

```
bool load(const char *dictionary)
{
    // Open the dictionary file

    // Read each word in the file

    // Add each word to the hash table

    // Close the dictionary file
}
```

Consider first how to open the dictionary file. `fopen` (<https://manual.cs50.io/3/fopen>) is a natural choice. You can use mode `r`, given that you need only *read* words from the dictionary file (not *write* or *append* them).

```
bool load(const char *dictionary)
{
    // Open the dictionary file
    FILE *source = fopen(dictionary, "r");

    // Read each word in the file

    // Add each word to the hash table

    // Close the dictionary file
}
```

Before moving on, you should write code to check whether the file opened correctly. That's up to you! It's also best to ensure you close every file you open, so now's a good time to write the code to close the dictionary file:

```
bool load(const char *dictionary)
{
    // Open the dictionary file
    FILE *source = fopen(dictionary, "r");

    // Read each word in the file

    // Add each word to the hash table

    // Close the dictionary file
    fclose(source);
}
```

What remains is to read each word in the file and to add each word to the hash table. Return `true` when the entire operation is successful and `false` if it ever fails. Consider following this problem's walkthrough and continue to break sub-problems into even smaller problems. For example, adding each word to the hash table might only be a matter of implementing a few, even smaller, steps:

1. Create space for a new hash table node
2. Copy the word into the new node
3. Hash the word to obtain its hash value
4. Insert the new node into the hash table (using the index specified by its hash value)

Of course, there's more one way to approach this problem, each with their own design trade-offs. For that reason, the rest of the code is up to you!

▼ Implement hash

Complete the `hash` function. `hash` should take a string, `word`, as input and return a positive ("unsigned") `int`.

The hash function given to you returns an `int` between 0 and 25, inclusive, based on the first character of `word`. However, there are many ways to implement a hash function beyond using the first character (or *characters*) of a word. Consider a hash function that uses a sum of ASCII values or the length of a word. A good hash function reduces “collisions” and has a (mostly!) even distribution across hash table “buckets”.

▼ Implement `size`

Complete the `size` function. `size` should return the number of words loaded in the dictionary. Consider two approaches to this problem:

- Count each word as you load it into the dictionary. Return that count when `size` is called.
- Each time `size` is called, iterate through the words in the hash table to count them up. Return that count.

Which seems most efficient to you? Whichever you choose, we’ll leave the code up to you.

▼ Implement `check`

Complete the `check` function. `check` should return `true` if a word is located in the dictionary, otherwise `false`.

Consider that this problem is also composed of smaller problems. If you’ve implemented a hash table, finding a word takes only a few steps:

1. Hash the word to obtain its hash value
2. Search the hash table at the location specified by the word’s hash value
 1. Return `true` if the word is found
3. Return `false` if no word is found

To compare two strings case-insensitively, you may find `strcasecmp` (<https://man.cs50.io/3/strcasecmp>) (declared in `strings.h`) useful! You’ll likely also want to ensure that your hash function is case-insensitive, such that `foo` and `F00` have the same hash value.

▼ Implement `unload`

Complete the `unload` function. Be sure to `free` in `unload` any memory that you allocated in `load`!

Recall that `valgrind` is your newest best friend. Know that `valgrind` watches for leaks while your program is actually running, so be sure to provide command-line arguments if you want

`valgrind` to analyze `speller` while you use a particular `dictionary` and/or text, as in the below. Best to use a small text, though, else `valgrind` could take quite a while to run.

```
valgrind ./speller texts/cat.txt
```

If you run `valgrind` without specifying a `text` for `speller`, your implementations of `load` and `unload` won't actually get called (and thus analyzed).

If unsure how to interpret the output of `valgrind`, do just ask `help50` for help:

```
help50 valgrind ./speller texts/cat.txt
```

Walkthroughs

Note that there are 6 videos in this playlist.



How to Test

How to check whether your program is outting the right misspelled words? Well, you're welcome to consult the "answer keys" that are inside of the `keys` directory that's inside of your `speller`

directory. For instance, inside of `keys/lalaland.txt` are all of the words that your program *should* think are misspelled.

You could therefore run your program on some text in one window, as with the below.

```
./speller texts/lalaland.txt
```

And you could then run the staff's solution on the same text in another window, as with the below.

```
./speller50 texts/lalaland.txt
```

And you could then compare the windows visually side by side. That could get tedious quickly, though. So you might instead want to “redirect” your program's output to a file, as with the below.

```
./speller texts/lalaland.txt > student.txt
./speller50 texts/lalaland.txt > staff.txt
```

You can then compare both files side by side in the same window with a program like `diff`, as with the below.

```
diff -y student.txt staff.txt
```

Alternatively, to save time, you could just compare your program's output (assuming you redirected it to, e.g., `student.txt`) against one of the answer keys without running the staff's solution, as with the below.

```
diff -y student.txt keys/lalaland.txt
```

If your program's output matches the staff's, `diff` will output two columns that should be identical except for, perhaps, the running times at the bottom. If the columns differ, though, you'll see a `>` or `|` where they differ. For instance, if you see

MISSPELLED WORDS	MISSPELLED WORD
TECHNO	TECHNO
L	L
Prius	> Thelonious
	Prius
	> MIA
L	L

that means your program (whose output is on the left) does not think that `Thelonious` or `MIA` is misspelled, even though the staff's output (on the right) does, as is implied by the absence of, say,

The `lonious` in the lefthand column and the presence of `The lonious` in the righthand column.

Finally, be sure to test with both the default large and small dictionaries. Be careful not to assume that if your solution runs successfully with the large dictionary it will also run successfully with the small one. Here's how to try the small dictionary:

```
./speller dictionaries/small texts/cat.txt
```

Correctness

```
check50 cs50/problems/2025/x/speller
```

Style

```
style50 dictionary.c
```

Staff's Solution

How to assess just how fast (and correct) your code is? Well, as always, feel free to play with the staff's solution, as with the below, and compare its numbers against yours.

```
./speller50 texts/lalaland.txt
```

How to Submit

```
submit50 cs50/problems/2025/x/speller
```

